

Application Security Review

By Aditi Chintarevula

Executive Summary

The Customer Information Management System (CIMS) is designed to help the company store and manage customer information efficiently. It allows customers to update their own data, customer relationship managers to handle client details internally, and data scientists to analyze usage trends and billing information.

During the review, I found a few serious security issues that could expose sensitive customer and payment information. The main risks include the absence of encryption in the database, exposure to SQL injection, hardcoded credentials in the internal web portal, open database access ports, and weak access control for data scientists.

*To address these issues, I recommend enabling encryption for data storage and transfer, securing credentials using a secret manager, implementing stronger access control, and isolating the database within a private network. These improvements not only strengthen overall security but also ensure compliance with **Sarbanes - Oxley [SOX]** and **Payment Card Data Security Standards [PCI DSS]** standards while maintaining business needs.*

Risk 1: Card Holder Data stored without Encryption in the Database

Risk Rating: *Critical*

Related Security Frameworks (if any): *PCI DSS*

Explanation of Risk

The Billing table stores full processed credit card information (cc_number, expiry_date) and the highly sensitive cvv_code. The PostgreSQL database is running in its default mode with no database encryption enabled. Storing the CVV code at all after authorization is an automatic failure of PCI DSS compliance, regardless of encryption. Storing the cardholder's information and related data without encryption means that any data breach will instantly expose all cardholder data in plain text. This is a severe threat to customer funds and privacy, and a violation that would trigger the highest level of PCI DSS penalties and compliance failure.

Recommendations

(Include at least two. Or, if only giving one, explain why that is the only feasible solution.)

1) Immediately Stop Storing the cvv_code and implement a Tokenization or Payment Gateway Vault solution for the cc_number.

Complexity: High (Requires integrating with a third-party tokenization service and significant application re-architecture).

Time: 4-8 weeks.

Money: High (Licensing and integration costs for a tokenization service)

2) Implementing Transparent Data Encryption (TDE) or using PostgreSQL's built-in pgcrypto extension for column-level encryption on the Billing table's remaining sensitive fields (e.g., cc_number).

Complexity: Medium (Configuration/setup, requires careful key management).

Time: 2-3 weeks for implementation and testing.

Money: Medium management system costs).(Internal personnel time, potential key

How does the recommendation mitigate the risk?

Eliminating the storage of the CVV code removes the mandatory PCI DSS violation (PCI DSS Requirement 3). Tokenization replaces the actual important data with a non-sensitive surrogate value (token), which drastically reduces the scope of the Cardholder Data Environment (CDE) and renders the data useless if breached.

This ensures that the PAN is protected at rest as required by PCI DSS Requirement 3. This control reduces the impact of a database compromise (such as an attacker obtaining a raw database dump) because the sensitive data is obfuscated

Risk 2: SQL Injection in Customer Update Feature

Risk Rating: *Critical*

Related Security Frameworks (if any): *PCI DSS, SOX ,OWASP Top 10(A03:2021 – Injection)*

Explanation of Risk

Customer Updates feature is implemented by wrapping customer-supplied information directly into a raw SQL query string on the front-end (portal server) and sending this query to the DB Update Processor via a POST request. The processor then relays this query directly to the PostgreSQL database. This process is the classic definition of a SQL Injection vulnerability. An attacker can manipulate their input to execute database commands, such as (UPDATE ...; DROP TABLE Billing;).

A successful exploit would allow an authenticated customer to read, modify, or delete any data within the CIMS database, including all customer records, organizational usage data, and critically, the entire Billing table containing unencrypted credit card numbers and CVV codes. This results in a full data compromise, leading to massive financial and reputational impact.

Recommendations

(Include at least two. Or, if only giving one, explain why that is the only feasible solution.)

1) Using Parameterized Queries (Prepared Statements). The user input should be treated only as data, never as executable code.

Complexity: Medium (Requires application code refactoring and thorough unit testing).

Time: 2-4 weeks.

Money: Moderate developer cost

2) Implement a Web Application Firewall (WAF) in front of the DB Update Processor in the AWS VPC.

Complexity: Low (Cloud-native WAFs are easy to deploy).

Time: 1-2 days configuration.

Money: Monthly recurring cloud cost (low to medium).

How does the recommendation mitigate the risk?

Parameterized queries separate the SQL command from user-supplied input. The database engine then treats the input strictly as variable data, preventing an attacker from injecting control characters or new commands, thereby eliminating the SQL Injection vulnerability at its source.

A WAF inspects all HTTP traffic for known attack patterns (like SQL injection signatures) before it reaches the application. This provides a crucial defensive layer, reducing the opportunity for the threat to exploit the vulnerability

Risk 3: Database Port 5432 is Exposed to Internet

Risk Rating: *High*

Related Security Frameworks (if any): *PCI DSS*

Explanation of Risk

A firewall exception was added to allow traffic on port 5432 (PostgreSQL) from the DB Update Processor's public IP. Although the processor is in a private VPC, the database that stores sensitive cardholder data is not properly isolated from other systems. Allowing open access from a cloud component connected to the public internet breaks the principle of least privilege. If the processor or AWS environment is compromised, attackers could directly access the database, violating PCI DSS network segmentation and data protection requirements.

Recommendations

(Include at least two. Or, if only giving one, explain why that is the only feasible solution.)

How does the recommendation mitigate the risk?

1) Restrict database access to private subnets or VPN connections only. Prevents exposure of the database to the public internet, reducing the likelihood of brute-force attacks, port scans, or unauthorized access.

Complexity: Medium

Time: 2-3 weeks

Cost: Low-Medium

Configure the PostgreSQL database to accept connections only from trusted internal networks or through a secure VPN tunnel. Disable any inbound rules that allow access from public IP addresses, including those belonging to the AWS DB Update Processor. This ensures that the database is only reachable through authenticated and encrypted channels within a controlled network boundary.

2) Implement strict network segmentation and firewall rules to isolate the Cardholder Data Environment (CDE).

Complexity: Medium

Time: 3-4 weeks

Cost: Medium

Create dedicated network zones for systems that store or process cardholder data. Use firewalls or security groups to strictly control traffic between the CDE and less trusted environments, such as public-facing web servers or cloud processors. Only allow the minimal set of required connections (e.g., from an approved application server) and block all others by default.

Risk 4: HardCoded Database Credentials

Risk Rating: *High*

Related Security Frameworks (if any): *PCI DSS*

Explanation of Risk

The AWS database update processor and internal CRM web form use hardcoded usernames and passwords to connect to the PostgreSQL database. If an attacker gains access to the source code or server, they can easily extract these credentials and access or modify sensitive customer data. This violates secure credential storage requirements under PCI DSS and creates a serious risk of unauthorized database access.

Recommendations

(Include at least two. Or, if only giving one, explain why that is the only feasible solution.)

1) Use a Secret Management System (e.g., AWS Secrets Manager or HashiCorp Vault) to store database credentials securely.

Complexity: Medium

Time: 2–3 weeks

Cost: Medium

2) Replace static credentials with IAM roles or short-lived access tokens.

Complexity: Low–Medium

Time: 1–2 weeks

Cost: Low

How does the recommendation mitigate the risk?

Storing credentials in a secret manager ensures that sensitive data never appears in plain text within code, repositories, or configuration files. Secrets are encrypted, access is logged, and permissions can be tightly controlled. Even if source code is leaked, attackers can't retrieve the credentials, effectively removing the main risk of hardcoded secrets.

Replacing fixed credentials with IAM roles or temporary tokens eliminates the need for long-lived passwords. Tokens automatically expire, which limits the damage if they are ever exposed. It also enforces continuous credential rotation and least-privilege access, greatly reducing the chance of unauthorized database access.

Final Architecture Recommendation

<i>Risks</i>	<i>Mitigating controls</i>	<i>Why this works</i>
1) Cardholder data stored without encryption (including CVV)	Stop storing CVV immediately. Integrate a PCI-compliant payment gateway or tokenization service. Encrypt data at rest and enable database-level key management with an external KMS	Tokenization removes sensitive card fields from the CDE scope and prevents CVV storage violations. Encryption at rest protects data if disks or backups are stolen.
2) SQL injection in customer update feature	Replace the current raw-SQL flow with an API layer: the portal submits structured requests to a DB Update API (hosted in AWS), which validates inputs and uses parameterized queries. Place a Web Application Firewall (WAF) in front of the portal and the DB Update API.	The API encapsulates SQL access and prevents untrusted input from becoming executable SQL. WAF and validation act as secondary protections for unexpected attack payloads.
3) Database port (5432) exposed & discoverable	Remove public firewall rules for port 5432. Only allow DB connections from internal subnets, dedicated VPC peering, or site-to-site VPN.	Eliminates public discoverability and direct attack vectors; limits access to authenticated, monitored entry points.
4) Hardcoded credentials	Move all credentials to a Secrets Manager (e.g., AWS Secrets Manager) applications fetch secrets at runtime. Where possible, use short-lived credentials or IAM roles for AWS-hosted components; for on-prem DB access consider federated identities or short-lived tokens minted by a trusted broker.	Removes secrets from source code and repositories, introduces audit trails, and limits the window of exposure.

How the proposed architecture still meets the business requirements :

- *Customer updates: Customers continue to update data using the same portal. Instead of sending raw SQL, the portal calls the DB Update API over HTTPS (TLS). The API validates inputs and performs safe, parameterized updates. UX is unchanged for customers.*
- *CRM updates: CRM tools call internal APIs (or connect via an authenticated, internal-only management interface) and operate with least-privilege accounts. Authorized CRMs retain the ability to search, update, and manage customer records.*

Cost Summary

Category	Estimated Cost	Effort / Time	Explanation
<i>1) Configuration / policy changes (enabling DB configuration, firewall rules)</i>	<i>Low</i>	<i>Low-Medium effort</i>	<i>Primarily setup and configuration; minimal downtime expected</i>
<i>2) Secret Management & IAM Role Changes</i>	<i>Medium</i>	<i>Medium effort</i>	<i>One-time integration with low maintenance cost</i>
<i>3) Tokenization / Payment Gateway Integration</i>	<i>High</i>	<i>High effort / Long term (6-8 weeks)</i>	<i>Requires significant re-architecture and compliance validation</i>
<i>4) WAF, IDS/IPS, SIEM & Monitoring</i>	<i>Medium (Recurring)</i>	<i>Medium effort</i>	<i>Adds ongoing monitoring and licensing costs.</i>

All recommended changes are incremental and compatible; none of the controls prevent other controls from being applied, so the architecture is logically consistent.

Unified Final Architecture Diagram:

